KBTU · SCHOOL OF IT AND ENGINEERING

AI-Driven Software Engineering

Lesson #01 — Lecture

Introduction to Software Engineering & Software Processes

Fall 2026 · 3 credits · 15 weeks

LESSON #01

Agenda

1. Course overview & organization
2. Course importance
3. Professional software development
4. Software engineering ethics
5. Software process models
6. Process activities
7. Coping with change
8. Process improvement

Items 1–4 · 10-minute break · Items 5–8 · then Practice #01

01

Course overview & organization

What this course is, how it runs, and how you are graded.

COURSE OVERVIEW

Goals & objectives

This course teaches software engineering in the age of AI: the same engineering responsibilities, carried out with a new class of tools.

I

Foundations of Software Engineering

- Processes, requirements, modelling, design
- Testing, evolution, dependability, security
- Reference: Ian Sommerville, 10th edition

II

The Modern Software Developer

- Coding agents, context engineering, agent skills
- AI review, security, background agents, teams
- Reference: Stanford CS146S

Weeks 1–5 build the engineering vocabulary. Weeks 6–15 apply it to AI-assisted development.

COURSE ORGANIZATION

Course details

3
credits

15
weeks of classes

45
class hours

2 + 1
hours per week
(lecture + practice)

- Academic year 2026–2027, Fall semester
- September 2026 – December 2026
- Lessons as scheduled in WSP; office hours Monday 09:00–15:50
- Announcements, materials and assignments: MS Teams and UNINET/WSP

COURSE ORGANIZATION

Your instructors

ALDAMURATOV Jomart

z.aldamuratov@kbtu.kz

AZIMKHANOV Bauyrzhan

b.azimkhanov@kbtu.kz

KARATAYEV Aron

aro_karatayev@kbtu.kz

KYDYRALIN Magzhan

m.kydyralin@kbtu.kz

Every group has its own MS Teams team — the code is posted in the syllabus and pinned in your channel.

COURSE ORGANIZATION

MS Teams class

CLASS CODE

`jl8fn4n`

Join once, in the first week. Everything else in this course starts here.

1. Open MS Teams → Teams → Join or create a team
2. Enter the code above and join the class team
3. Turn notifications on for the Announcements channel

What lives in Teams

- Announcements — schedule changes, deadlines, everything official
- Lecture slides and practice instructions, week by week
- Weekly assignments — you submit the link to your open pull request here
- The GitHub username registration form (do this today)
- Your questions — ask in the channel, not by direct message

COURSE ORGANIZATION

Part I — Foundations (weeks 1–5)

W1	Professional software development, ethics, software processes, coping with change
W2	Agile methods and techniques · Requirements engineering, elicitation, validation
W3	System modelling · Architectural design · Design and implementation, UML, patterns
W4	Development testing, TDD, release and user testing · Evolution, legacy systems
W5	Dependability, reliability, safety, security · Reuse, distributed and real-time systems

Part II — The Modern Software Developer (weeks 6–15)

W6 What an LLM actually is · The agent loop, tools and system prompts

W7 Advanced prompting · RePPIT and spec-driven development · MCP fundamentals

W8 Agent skills (SKILL.md) · Extending agents beyond the repo · Working from the CLI

W9 CLAUDE.md / AGENTS.md · Hooks and guardrails · Planner–implementer–reviewer subagents

W10 Agent-ready codebases: structure, docs, tests, readiness scoring

W11 AI code review: what it catches, what it misses, fitting it into a PR workflow

W12 SAST/SCA, dependency and secret leaks · Prompt injection · Agent attack surfaces

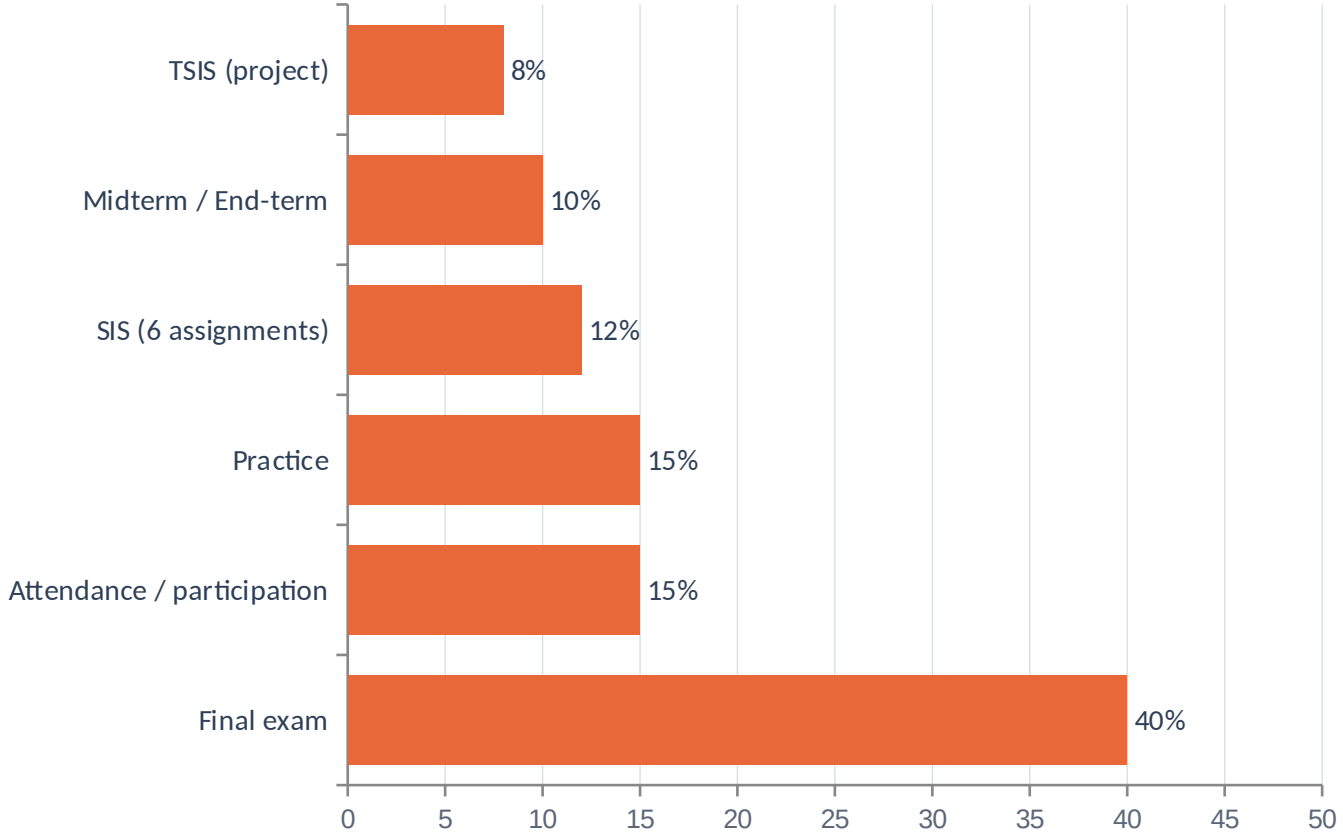
W13 Async cloud agents · Fleets of parallel agents · Issue-to-PR pipelines

W14 MCP portals · LLM gateways, routing and cost · Org-wide adoption (project defense)

W15 Self-improving systems · Securing agents post-deployment · What comes next

COURSE ORGANIZATION

How you are assessed



Continuous, not final

60% of your grade is earned before the exam — one point of attendance and one point of practice every single week.

Two hard rules

- Miss more than 30% of lessons → F
- Score ≤ 9.4 on the final exam → F, whatever else you did

COURSE ORGANIZATION

SIS and TSIS schedule

Six independent assignments build one small software application, milestone by milestone. The supervised project (TSIS) is its demonstration and defense.

W2	W4	W6	W8	W10	W12	W14
SIS #01	SIS #02	SIS #03	SIS #04	SIS #05	SIS #06	TSIS
2 points	2 points	2 points	2 points	2 points	2 points	8 points

- Requirements → architecture → implementation → AI-assisted development → testing and security → documentation
- Everything is graded on evidence: working code, tests, diagrams, and your own explanation of it
- Week 14 is a supervised demonstration and an individual defense — you answer for every line you submit

COURSE ORGANIZATION

How you submit work

1

One repository for the semester

se-practice on GitHub. Register your GitHub username through the form in Teams in week 1.

2

One branch and folder per week

Branch week-nn, folder week-nn/, at least three meaningful commits. Never commit to main.

3

One open pull request

Open a PR week-nn → main in your own repository and leave it open. A merged PR is not a submission.

4

The PR link goes in Teams

Paste the pull request link into that week's assignment, together with AI_USAGE.md in the week folder.

COURSE ORGANIZATION

AI use policy

A	**AI prohibited**	Generative AI tools are not permitted.
B	**Limited use**	Brainstorming, concept explanation, language improvement only.
C	**AI-assisted**	Support for writing, debugging and feedback; the main work is yours.
D	**AI-integrated**	AI may generate text, code, analysis and solutions. You remain fully responsible.

This course runs at Level D — and Level D has a price: disclose the tools you used, and be able to defend every line you submit.

COURSE OVERVIEW

By the end of the course you can...

Build

Design, implement, test, document and demonstrate a small application using LLM-based tools — and explain every generated line.

Specify

Formulate requirements, constraints, acceptance criteria and repository context for AI-assisted development.

Verify

Check AI-generated software with automated tests, static analysis, code review and security analysis.

Control

Configure bounded agent workflows: permissions, checkpoints, observability, recovery.

Compare

Weigh conventional against AI-assisted workflows on quality, effort, cost and reliability.

Answer for it

Identify privacy, licensing and security risks, document AI use, and defend your engineering decisions.

COURSE ORGANIZATION

References

Software Engineering, Global Edition, 10th ed.

Ian Sommerville, 2016, Pearson. Main textbook for weeks 1–5.

CS146S — The Modern Software Developer

Mihail Eric, Stanford University. Basis for weeks 6–15.

UML Distilled, 3rd ed. (optional)

Martin Fowler, 2004, Addison-Wesley. For modelling and diagrams in practice.

Design Patterns (optional)

Gamma, Helm, Johnson, Vlissides, 1994, Addison-Wesley. For refactoring generated code.

COURSE ORGANIZATION

Why attendance is not a formality

20%

of the knowledge is in the slides

80%

arrives through discussion, worked examples, review and the questions you ask

Participation ≠ presence

Read the material before class. Come with a question. One point every week depends on it.

The lecture is where you learn to judge software. The practice is where you find out whether you can.

02

Course importance

Why this subject, and why now.

COURSE IMPORTANCE

Four generations, one lifetime

1990 – 2000

PC

Software on a desk. Shipped on disks, updated once a year.

2000 – 2010

Internet

Software as a service. Deployed continuously, used by millions.

2010 – 2020

Smartphone

Software in a pocket. Sensors, location, always connected.

2020 – 2030

IoT & AI

Software everywhere — and software that writes software.

You are the first cohort of engineers whose main tool can produce a plausible answer to almost any question — including a wrong one.

COURSE IMPORTANCE

What AI actually changed

Collapsed to almost zero

- Time to produce a first working version
- Cost of boilerplate, glue code, scaffolding
- Cost of trying a second approach

Unchanged, or worse

- Cost of shipping something wrong
- Cost of code nobody understands
- Cost of a breach, an outage, a lawsuit

When writing code becomes cheap, the scarce skill is deciding what is correct.

The paradigm shifted from writing code to reading code — and reading it critically.

COURSE IMPORTANCE

Three questions to keep asking

WHY?

Why am I studying this?

Because you will be responsible for systems you did not fully write.

WHAT?

What am I studying?

Engineering — specification, design, verification, evolution — not just coding.

HOW?

How am I studying?

By building, breaking, comparing and defending your own work every week.

03

Professional software development

What software engineering is, what it costs, and why projects fail.

What software engineering is

> An engineering discipline concerned with all aspects of software production, from the early stages of system specification through to maintaining the system after it has gone into use.

- We cannot run the modern world without software: government, banking, energy, transport, health
- National infrastructure and utilities are controlled by computer-based systems
- Most electrical products contain a computer and controlling software
- Spending on software is a significant fraction of GNP in every developed country

Software costs

> 50%

of total system cost is software, not hardware

2–4×

maintenance cost versus development cost, over a long system life

~60%

of maintenance effort is understanding code before changing it

- Software engineering is concerned with cost-effective development, not with clever code
- A system that is cheap to write and expensive to change is a bad engineering trade
- AI tools reduce the cost of the first version — the smallest part of the total bill

Why software projects fail

1

Growing system complexity

New systems are bigger and more connected than the ones they replace, and they must be built faster.

2

Failure to use engineering methods

It is easy to write programs without using software engineering methods. Most large projects that do this fail.

3

New: unexamined generated code

Code that arrives faster than anyone can review it. Plausible, running, and wrong in ways nobody has checked.

Two of these are from 2016. The third is yours to deal with.

Two kinds of software product

Generic products

- Stand-alone systems sold to any buyer
- The developer owns the specification
- Examples: databases, CAD, office suites
- Change is decided by the vendor

Customized products

- Commissioned by one specific customer
- The customer owns the specification
- Examples: control systems, air traffic, bank back-office
- Change is negotiated with the customer

The distinction decides who can change the requirements — and therefore how you must run the process.

PROFESSIONAL SOFTWARE DEVELOPMENT

Essential attributes of good software

1 Acceptability

Understandable, usable and compatible with the systems users already have.

2 Dependability & security

It does not fail dangerously, and it does not let others cause harm.

3 Efficiency

It does not waste resources: memory, processor cycles, energy, response time.

4 Maintainability

It can be changed by other people as requirements change. This is the expensive one.

AI tools optimize for 'it runs'. None of the four attributes above is checked unless a human checks it.

Software engineering diversity

- Stand-alone applications
- Interactive transaction-based applications
- Embedded control systems
- Batch processing systems
- Entertainment systems
- Modelling and simulation systems
- Data collection and analysis systems
- Systems of systems

There is no universal method. The right process depends on the kind of system, its customer, and what failure would cost.

04

Software engineering ethics

Competence, confidentiality, ownership — and who answers for generated code.

Four classic obligations

Confidentiality

Respect the confidentiality of your employer and clients, whether or not you signed anything.

Competence

Do not accept work that is beyond your competence, and do not misrepresent your level of skill.

Intellectual property rights

Know and respect the law on patents, copyright and licences — and your employer's rights.

Computer misuse

Do not use your technical skills to misuse other people's computers or data.

Software engineering happens inside a legal and social framework. Being technically right is not a defence.

The same four, in the age of AI

Confidentiality → prompts leak	Pasting client code, credentials or personal data into a hosted model is a disclosure. Treat the prompt box as a public channel.
Competence → borrowed fluency	A tool that writes confident code does not make you competent in that language, framework or domain. Do not claim what you cannot defend.
IP rights → unclear provenance	Generated code can carry licence obligations you never read. Dependencies suggested by a model still need checking.
Misuse → capability at scale	The same tools that write your tests can write a scraper, a phishing page, or an exploit. Intent is your responsibility.

Disclosure is not a bureaucratic step. It is how a profession keeps track of who is accountable for what.

SOFTWARE ENGINEERING ETHICS

Discussion — where is the line?

Case A

You paste your employer's authentication module into a public AI tool to ask why a test fails.

Case B

An agent adds a dependency you have never heard of. The build is green. You merge it.

Case C

You submit an assignment that an AI wrote end to end, and you can explain every line of it.

For each case: is it wrong, risky, or fine? What would you have to know to decide?

Vote by show of hands, then argue. There is a defensible answer on both sides of Case C.

END OF PART ONE

10-minute break

Resume at: __ : __

Think about it over the break

Of the three cases we just discussed, which one would you be willing to defend — out loud, to the client whose code it was?

Just covered

What software engineering is · what it costs · why projects fail · the four attributes · ethics, classic and AI-era

Coming up

Software processes and models · process activities · coping with change · process improvement

Before you go: check that you can open your MS Teams channel and see Practice #01.

05

Software process models

The shapes a development process can take.

SOFTWARE PROCESSES

A process is a set of related activities

A software process is a set of related activities that leads to the production of a software system.

1

Specification

What the software must do, and the constraints on how it operates.

2

Development

Designing and programming software that meets the specification.

3

Validation

Checking that it does what the customer actually wants.

4

Evolution

Changing it as customer needs change — for as long as it is used.

PROCESS MODELS

The waterfall model

1. Requirements analysis and definition
2. System and software design
3. Implementation and unit testing
4. Integration and system testing
5. Operation and maintenance

Use it when

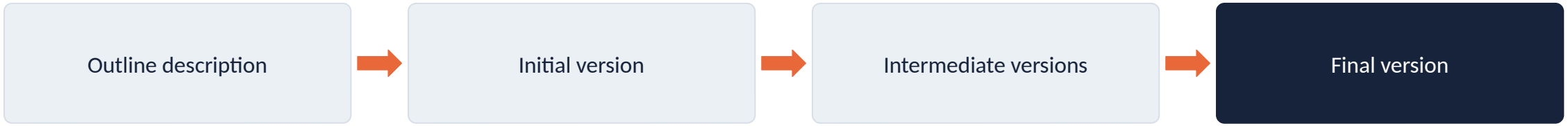
Requirements are stable and understood, several sites must be coordinated, or a regulator requires documented sign-off at each stage.

The cost

A phase ends before the next begins, so change late in the project is expensive. Few real projects follow it strictly.

Incremental development

Specify, develop and validate a little at a time, in versions, until the system is adequate.



Why it wins

- Cost of changing requirements falls
- Customer feedback arrives early
- Working software is delivered sooner

What it costs

- The process is less visible to managers
- Structure degrades unless money is spent on refactoring
- Documentation is often skipped

Integration and configuration

Build the system by configuring and composing existing components, services and off-the-shelf systems, instead of writing everything from scratch.

1. **Requirements specification**
2. **Software discovery and evaluation**
3. **Requirements refinement**
4. **Configuration / adaptation**
5. **Development and integration**

Reduced cost and risk — at the price of control over what you shipped

Using a code-generating model is a form of reuse. The same question applies: do you know what you have taken on?

Which model, when?

Requirements are stable and must be signed off	**Waterfall**
Requirements will change during development	**Incremental**
A suitable component or service already exists	**Integration & configuration**
The customer cannot describe what they want yet	**Prototyping first, then incremental**
Failure would cost lives or licences	**Waterfall-like, with heavy validation**

In practice, most large systems mix all three. Choosing consciously is the engineering part.

06

Process activities

What actually happens inside specification, design, validation and evolution.

PROCESS ACTIVITIES

Specification, then design and implementation

Software specification

- Requirements elicitation and analysis
- Requirements specification
- Requirements validation
- Output: a document everyone agrees on — and can be tested against

Design and implementation

- Architectural design — the overall structure
- Database design — the data and its structure
- Interface design — how components talk to each other
- Component selection and design — buy, reuse, or build

PROCESS ACTIVITIES

Validation, then evolution

1. **Component testing**
2. **System testing**
3. **Customer / acceptance testing**

Verification asks: are we building the product right? Validation asks: are we building the right product?

Software evolution

Software is flexible, and it changes. Requirements shift as business changes, so the system must change with them. The old separation between 'development' and 'maintenance' is artificial: only the very smallest systems are built once and never touched again.

07

Coping with change

Change is not a failure of planning. It is the environment.

COPING WITH CHANGE

Two strategies against the cost of change

Change avoidance

- Anticipate likely changes before serious work begins
- Prototype to test ideas cheaply
- Get the requirements wrong on paper, not in production

Change tolerance

- Design the process so change is cheap to absorb
- Deliver in increments; change affects the next one, not all of it
- Automate the checks that make change safe

Rework is the cost of change. Every technique in this course is, in the end, a way of making rework cheaper.

Prototyping and incremental delivery

Prototyping

- An early version used to demonstrate concepts and try out options
- Supports requirements elicitation and design validation
- Ignores non-functional requirements: no error handling, no security, weak quality
- Must be thrown away, or knowingly hardened

Incremental delivery

- Increments are delivered to the customer and put into use
- Earlier increments get real feedback from real users
- Highest-priority requirements go first, so they get the most testing
- Hard when the new system replaces one that must switch over at once

08

Process improvement

Making the process cheaper, faster and better — deliberately.

PROCESS IMPROVEMENT

Understand, then change

Process improvement means understanding existing processes and changing them to increase product quality, reduce cost, or shorten development time.

1

Measure

Collect data about the process and the product. You cannot improve what you never measured.

2

Analyse

Find where the goals are not being met and what causes it.

3

Change

Introduce the change, then measure again to see whether it worked.

Industry demands cheaper and better software on tighter deadlines. Improvement is how organisations answer that without simply working longer.

PROCESS IMPROVEMENT

Two ways to improve

The process maturity approach

- Improve process management and introduce good practice
- Aim: predictability, repeatability, quality control
- Fits large organisations and long-lived systems

The agile approach

- Iterate fast, cut overhead, reduce response time
- Aim: speed and responsiveness to change
- Fits small teams and uncertain requirements

Neither is universally right. Ask what your project would lose if you chose the other one.

TODAY'S PRACTICE

Practice #01 — AI in Software Engineering: First Contact

1

Build it by hand

A small program over a list of student marks: average, highest, lowest, pass rate. No AI in this part.

2

Build it with an AI tool

Same task, one plain-language prompt in Rocket. Record every prompt, question and answer.

3

Compare and defend

Test both against the same data, fill in comparison.md and AI_USAGE.md, and say which one you would sign.

- Submit: branch week-01, folder week-01/, at least three commits, an open pull request — the PR link goes into the Teams assignment
- Full instructions are in week-01/README.md; repository setup is in SETUP.md
- AI is forbidden in part 1 and expected in part 2. Both must be disclosed honestly.

LESSON #01

What to take away

1. **Software engineering is about all of it** — Specification, design, validation and evolution — not only the part where code appears.
2. **Maintenance dominates cost** — Code that is cheap to produce and expensive to understand is a bad trade, whoever wrote it.
3. **The process is a choice** — Waterfall, incremental, integration — pick deliberately, based on the system and its risks.
4. **Change is the normal case** — Avoid it with prototypes, tolerate it with increments and tests.
5. **AI changes the cost, not the duty** — The engineer still answers for correctness, security, maintainability — and for disclosure.

Q & A

Next: Lesson #02 — Agile methods and requirements engineering

Before then: Practice #01, submitted as an open pull request in your se-practice repository.